

Estructuras de Datos y Algoritmos

Tema 1: Tipos de Datos Abstractos

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Definición: Abstracción

La abstracción es un proceso mental que consiste en aislar propiedades o conceptos esenciales de un objeto, separándolo de sus detalles particulares y del contexto. Es la capacidad que nos permite comprender ideas generales y formular conceptos, dejando de lado los rasgos secundarios.

Ejemplos de ello se encuentran en:

- Filosofía y psicología
- Arte
- Ciencias de la computación
- Lingüística

Concepto de abstracción en ciencias de la computación

Definición: Abstracción en ciencias de la computación

Es el principio fundamental que consiste en ocultar detalles complejos y mostrar únicamente las características o funcionalidades esenciales de un objeto o sistema.

Se pueden abstraer:

- Datos: consiste en separar el uso de un dato complejo de los detalles internos de cómo está construido.
- Procedimientos: consiste en ocultar los pasos internos de un procedimiento detrás de un bloque ejecutable.

Dato y tipo de dato

Definición: Dato

Unidad de información que un programa puede representar, almacenar, transmitir o procesar.

Ejemplos: 42, 3.5, 'Ana' y True.

- `7 + 3` produce 10 porque los valores son `int`;
- `'7' + '3'` produce `'73'` porque son `str`.

El tipo determina cómo interpretar cada dato y qué operaciones son válidas.

Definición: Tipo de dato

Determina el conjunto de valores posibles y las operaciones que tienen sentido sobre ellos.

Tipos incorporados y definidos por el usuario

Los tipos pueden distinguirse por su origen:

- **Incorporados:** ya forman parte de Python.
- **Definidos por el usuario:** los crea el programador para representar conceptos propios del problema.

Los incorporados:

- Numéricos: `int` y `float`.
- Lógico: `bool`, cuyos valores son `True` y `False`.
- Texto: `str`.
- Colecciones: agrupan varios valores `list`, `tuple`, `dict` y `set`.

Los definidos por el usuario:

- Al declarar `class Contador:`, definimos un nuevo tipo de dato llamado `Contador`. Sus atributos representan el estado y sus métodos determinan las operaciones disponibles.

Estructuras de datos

Definición: Estructuras de datos

Una estructura de datos (ED) es una representación organizada de un conjunto de información.

Estas estructuras de datos pueden ser:

- Lineales: sus elementos forman una sola secuencia.
- No lineales: sus elementos no forman una única secuencia posicional; pueden organizarse de manera asociativa, jerárquica o interconectada.

Además, según si su estado puede cambiar después de su creación:

- Mutables: permiten modificar el mismo objeto. Ejemplos: `list`, `dict` y `set`.
- Inmutables: su estado no puede modificarse; una operación produce un objeto nuevo. Ejemplos: `tuple` y `str`.

Operaciones frecuentes (dependen de la estructura):

- Consulta: acceder, buscar o recorrer elementos.
- Transformación: insertar, eliminar o modificar elementos.

Estructuras de datos: ejemplos cercanos

ED	Ejemplo	Organización y acceso	Mutabilidad
list	scores = [8, 9, 7]	Lineal, por posición	Mutable
tuple	point = (10, 20)	Lineal, por posición	Inmutable
dict	scores = {'Ana': 8, 'Luis': 9}	No lineal, por clave	Mutable
set	colors = {'rojo', 'azul'}	No lineal, por pertenencia	Mutable

Tipo de dato y estructura de datos

list, tuple, dict y set son tipos de dato incorporados de Python. Como organizan colecciones de valores, también pueden considerarse estructuras de datos concretas.

Tipo de dato abstracto

Definición: Tipo de dato abstracto (TDA)

Un TDA especifica los estados válidos, las operaciones disponibles y las reglas que determinan su comportamiento, sin establecer cómo se representa o implementa.

El TDA especifica qué:

- Estados válidos.
- Operaciones disponibles.
- Resultados y restricciones.

La implementación decide cómo:

- Representar el estado.
- Organizar los datos.
- Programar las operaciones.

Un mismo TDA puede tener diferentes implementaciones.

Ejemplo inicial: TDA Contador

Contador de un número entero no negativo: $valor \in \mathbb{Z}_{\geq 0}$.

Operación	Comportamiento esperado
<code>__init__()</code>	Inicializa el valor en 0.
<code>increment()</code>	Aumenta el valor en 1.
<code>decrement()</code>	Disminuye el valor en 1, si es posible.
<code>reset()</code>	Regresa el valor a 0.
<code>value()</code>	Devuelve el valor actual sin modificarlo.

Precondiciones y postcondiciones

Precondición

Lo que debe cumplirse antes de ejecutar una operación.

Postcondición

Lo que debe cumplirse después de ejecutar una operación, si la precondición era válida.

Operación	Precondición	Postcondición
<code>__init__()</code>	ninguna	$valor = 0$
<code>increment()</code>	ninguna	$valor' = valor + 1$
<code>decrement()</code>	$valor > 0$	$valor' = valor - 1$
<code>reset()</code>	ninguna	$valor' = 0$
<code>value()</code>	ninguna	no cambia el estado

Implementación en Python: primera versión

```
class Counter:
    def __init__(self):
        self._value = 0

    def increment(self):
        self._value += 1

    def decrement(self):
        if self._value == 0:
            raise ValueError("El contador no puede ser negativo")
        self._value -= 1

    def reset(self):
        self._value = 0

    def value(self):
        return self._value
```

Qué representa cada parte

Elemento en Python	Interpretación en el TDA
<code>class Counter</code>	Implementación concreta del TDA.
<code>self._value</code>	Representación interna del estado.
<code>increment, decrement</code>	Operaciones de la interfaz.
<code>ValueError</code>	Manejo de una precondition violada.
<code>value()</code>	Consulta que no modifica el estado.

Observación

El usuario del contador no necesita saber cómo se guarda internamente el valor.

Clase, objeto y estado

Clase

Plantilla que define atributos y métodos.

```
class Counter
```

Objeto

Instancia concreta de una clase.

```
c = Counter()
```

Estado

Información interna en un momento dado.

```
c._value
```

En un TDA mutable, las operaciones pueden cambiar el estado del objeto.

Uso de un objeto

```
c = Counter()

c.increment()
c.increment()
print(c.value())    # 2

c.decrement()
print(c.value())    # 1

c.reset()
print(c.value())    # 0
```

Lectura abstracta

El usuario invoca operaciones; no manipula directamente la representación interna.

Interfaz pública vs. representación interna

Interfaz pública

Lo que el usuario debe usar.

- `increment()`
- `decrement()`
- `reset()`
- `value()`

Representación interna

Lo que la clase usa para funcionar.

- `_value`
- detalles de almacenamiento
- decisiones de implementación

Separar interfaz y representación reduce errores y permite modificar el código.

Segundo ejemplo: TDA Intervalo cerrado

Un intervalo cerrado $[a, b]$ contiene los valores comprendidos entre dos límites, con $a, b \in \mathbb{R}$ y $a \leq b$.

Operación	Comportamiento esperado
<code>Interval(a, b)</code>	Crea el intervalo cerrado $[a, b]$.
<code>contains(x)</code>	Indica si $x \in [a, b]$.
<code>length()</code>	Devuelve la longitud del intervalo.
<code>shift(delta)</code>	Transforma $[a, b]$ en $[a + \delta, b + \delta]$.

Invariante

La relación $a \leq b$ debe conservarse después de cada operación.

TDA Intervalo: precondiciones y postcondiciones

En la tabla, a y b describen el estado anterior; a' y b' , el estado posterior.

Operación	Precondición	Postcondición
Interval(a , b)	$a \leq b$	El nuevo intervalo representa $[a, b]$.
contains(x)	ninguna	Devuelve $a \leq x \leq b$; $a' = a$ y $b' = b$.
length()	ninguna	Devuelve $b - a$; $a' = a$ y $b' = b$.
shift(δ)	ninguna	$a' = a + \delta$ y $b' = b + \delta$.

Sumar la misma cantidad a ambos límites conserva la relación $a' \leq b'$.

TDA Intervalo: implementación en Python

```
class Interval:
    def __init__(self, lower, upper):
        if lower > upper:
            raise ValueError("Limites iniciales invalidos")
        self._lower = lower
        self._upper = upper

    def contains(self, value):
        return self._lower <= value <= self._upper

    def length(self):
        return self._upper - self._lower

    def shift(self, delta):
        self._lower += delta
        self._upper += delta
```

TDA Intervalo: qué representa cada parte

Elemento en Python	Interpretación en el TDA
<code>class Interval</code>	Implementación concreta del TDA.
<code>_lower, _upper</code>	Representación interna de los límites.
<code>contains(), length()</code>	Consultas que no modifican el estado.
<code>shift()</code>	Transformación que conserva $a \leq b$.
<code>raise ValueError(...)</code>	Señala límites iniciales inválidos.

Observación

El usuario consulta pertenencia y longitud o desplaza el intervalo sin manipular directamente su representación interna.

Uso de un intervalo

```
interval = Interval(2, 5)

print(interval.contains(4))    # True
print(interval.length())      # 3

interval.shift(-2)             # ahora representa [0, 3]
print(interval.contains(0))    # True
print(interval.contains(4))    # False
print(interval.length())      # 3
```

Lectura abstracta

`shift()` cambia el estado mediante la interfaz y conserva tanto el orden de los límites como la longitud del intervalo.

Ejercicio: suma de vectores

Completa el método `add()` para calcular la suma de dos vectores:

$$[1, 2, 3] + [4, 5, 6] = [5, 7, 9].$$

Contrato

Los vectores deben tener la misma dimensión y el método debe devolver un nuevo vector sin modificar los originales.

```
class Vector:
    def __init__(self, data):
        self._data = data[:]

    def add(self, other):
        # Completa este metodo
```

```
v1 = Vector([1, 2, 3])
v2 = Vector([4, 5, 6])
result = v1.add(v2)  # nuevo Vector: [5, 7, 9]
```

Respuesta: suma de vectores

```
def add(self, other):
    if len(self._data) != len(other._data):
        raise ValueError("Las dimensiones deben coincidir")

    result = []
    for i in range(len(self._data)):
        value = self._data[i] + other._data[i]
        result.append(value)

    return Vector(result)
```

Encapsulamiento

Definición: Encapsulamiento

Encapsular significa ocultar los detalles internos de una implementación y exponer solo una interfaz clara.

El TDA debe proteger su invariante: el contador no puede ser negativo.

En Python, el guion bajo no prohíbe el acceso, pero comunica que el atributo es interno.

Rompe la abstracción

```
c = Counter()
c._value = -1
print(c.value())    # estado invalido
```

Respetar la interfaz

```
c = Counter()
c.decrement()       # ValueError
print(c.value())    # sigue valido
```

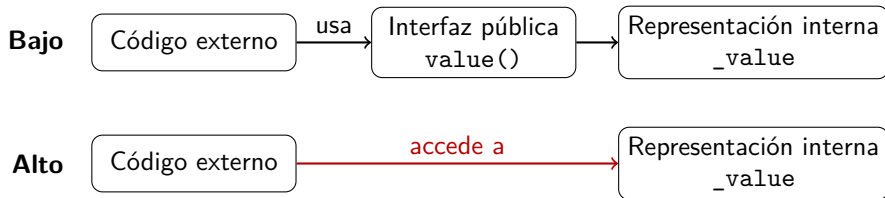
Lectura

La interfaz pública controla cómo cambia el estado interno.

Acoplamiento

Definición: Acoplamiento

El acoplamiento mide cuánto depende un componente de las decisiones de otro componente.



Bajo acoplamiento no significa ausencia de dependencia: el código que usa el TDA depende de una **interfaz estable**, no de los detalles internos.

Acoplamiento

La representación cambia, pero la interfaz pública conserva `value()`.

Versión 1

```
self._value = 0

def value(self):
    return self._value
```

Versión 2

```
self._units = []

def value(self):
    return len(self._units)
```

Código que usa el TDA	Depende de	V1	V2
<code>print(c._value)</code>	Representación interna	Funciona	Falla
<code>print(c.value())</code>	Interfaz pública	Funciona	Funciona

Si cambiar una decisión interna obliga a modificar el código que usa el TDA, existe **alto acoplamiento con la implementación**.

Acoplamiento

El mismo TDA Point puede guardar sus coordenadas de distintas formas sin cambiar el método público `distance_to_origin()`.

Representación cartesiana

```
self._x = x
self._y = y

def distance_to_origin(self):
    return (self._x**2 +
            self._y**2)**0.5
```

Representación polar

```
self._r = hypot(x, y)
self._theta = atan2(y, x)

def distance_to_origin(self):
    return self._r
```

La abstracción no cambia

En ambos casos, el código que usa el TDA solicita la distancia mediante la misma operación pública.

Acoplamiento: ¿qué código resiste el cambio?

Ambos fragmentos muestran la distancia al origen, pero dependen de cosas diferentes.

Alto acoplamiento

```
def show_distance(p):  
    d = (p._x**2 + p._y**2)**0.5  
    print(d)
```

Depende de que el punto use coordenadas cartesianas. Al cambiar a la representación polar, deja de funcionar.

Bajo acoplamiento

```
def show_distance(p):  
    print(p.distance_to_origin())
```

Depende sólo de la interfaz pública. Funciona con cualquiera de las dos representaciones.

Idea clave

El código con bajo acoplamiento no necesita conocer cómo se guarda el estado del punto.

Definición: Cohesión

La cohesión mide qué tan enfocada está una clase o módulo en una responsabilidad clara.

Alta cohesión

Una clase Counter solo maneja el contador: incrementar, decrementar, reiniciar y consultar.

Baja cohesión

La clase también lee archivos, imprime menús, grafica resultados y pide entrada al usuario.

Diseño recomendado: **alta cohesión**.

Actividad: ¿qué pertenece a Counter?

Responsabilidad principal

Counter representa un contador no negativo y controla sus cambios.

Consideren las siguientes operaciones:

<code>increment()</code>	<code>ask_user_input()</code>	<code>value()</code>
<code>plot_counter()</code>	<code>decrement()</code>	<code>save_results()</code>
<code>is_zero()</code>	<code>print_menu()</code>	<code>reset()</code>
<code>run_experiment()</code>	<code>can_decrement()</code>	<code>test_increment()</code>

- 1 Identifiquen cuáles contribuyen directamente a la responsabilidad principal.
- 2 Expliquen por qué las demás deberían estar fuera de Counter.
- 3 Si cambia la interfaz de usuario o la forma de graficar, ¿debería modificarse el TDA?

Archivo o clase	Responsabilidad
counter.py	Implementa el TDA contador.
test_counter.py	Verifica el comportamiento esperado.
main.py	Usa el contador en un programa.
experiments.py	Ejecuta experimentos o mediciones.

Principio

Cada componente debe tener una responsabilidad principal y bien delimitada.

Las pruebas deben derivarse de las operaciones y reglas del TDA.

Regla especificada	Prueba esperada
Un contador inicia en 0.	<code>Counter().value() == 0</code>
<code>increment()</code> aumenta en 1.	después de dos incrementos, valor 2
<code>reset()</code> deja el valor en 0.	valor 0 después de reiniciar
No puede ser negativo.	decrementar en 0 produce error

La instrucción `assert`

`assert condicion` expresa un resultado esperado. Si la condición es verdadera, la prueba continúa; si es falsa, Python produce un `AssertionError`. `pytest` detecta ese error y reporta qué expectativa no se cumplió.

Pruebas con pytest

```
import pytest
from counter import Counter

def test_counter_starts_at_zero():
    c = Counter()
    assert c.value() == 0

def test_increment_twice():
    c = Counter()
    c.increment()
    c.increment()
    assert c.value() == 2

def test_decrement_below_zero_raises_error():
    c = Counter()
    with pytest.raises(ValueError):
        c.decrement()
```

Actividad: pruebas

La implementación contiene un error, pero la prueba termina exitosamente.

Implementación defectuosa

```
def decrement(self):
    self._value -= 1
    if self._value < 0:
        raise ValueError(
            "Valor negativo"
        )
```

Prueba existente

```
def test_decrement_at_zero():
    c = Counter()
    with pytest.raises(ValueError):
        c.decrement()
```

Preguntas

¿Por qué pasa la prueba? ¿En qué estado queda el objeto? ¿Qué verificación hace falta?

Prueba más fuerte

```
def test_decrement_at_zero():  
    c = Counter()  
    with pytest.raises(ValueError):  
        c.decrement()  
  
    assert c.value() == 0
```

Implementación corregida

```
def decrement(self):  
    if self._value == 0:  
        raise ValueError(  
            "Valor negativo"  
        )  
    self._value -= 1
```

Conclusión

Comprobar una excepción no basta: también debemos verificar el estado observable después del intento fallido.

Actividad

La implementación contiene un error, pero la prueba termina exitosamente.

Implementación defectuosa

```
def reset(self):  
    self._value == 0
```

Prueba existente

```
def test_reset():  
    c = Counter()  
    c.reset()  
  
    assert c.value() == 0
```

Preguntas

¿Por qué pasa la prueba si `reset()` no modifica el estado? ¿Qué preparación hace falta para observar el efecto de la operación? ¿Cómo mejorarían la prueba?

Prueba más fuerte

```
def test_reset():  
    c = Counter()  
    c.increment()  
    c.increment()  
    assert c.value() == 2  
  
    c.reset()  
  
    assert c.value() == 0
```

Implementación corregida

```
def reset(self):  
    self._value = 0
```

Conclusión

Como el contador ya inicia en cero, la prueba original no permite observar el efecto de `reset()`. Para probar una operación, primero debemos construir un estado donde su efecto sea observable.

Representación interna e invariantes

Invariante de representación

Propiedad que debe cumplirse siempre para que el estado interno sea válido.

Para Counter, una invariante natural es:

$$_value \geq 0$$

Uso

Las operaciones deben preservar la invariante. Si una operación la rompe, la implementación no respeta el TDA.

Maximo Comun Divisor: definición y teorema

Para $d, a \in \mathbb{Z}$, con $d \neq 0$:

$$d \mid a \iff \exists k \in \mathbb{Z} : a = kd.$$

Definición

Para $a, b \in \mathbb{Z}$, no ambos cero, escribimos $d = \text{mcd}(a, b)$ si

$$d \in \mathbb{Z}^+ \wedge d \mid a \wedge d \mid b \wedge \forall c \in \mathbb{Z}^+ : (c \mid a \wedge c \mid b) \Rightarrow c \leq d.$$

Teorema: invarianza del MCD

Sean $a, b \in \mathbb{Z}$, con $b > 0$. Si

$$\exists q, r \in \mathbb{Z} : a = qb + r \quad \wedge \quad 0 \leq r < b,$$

entonces

$$\text{mcd}(a, b) = \text{mcd}(b, r).$$

Aquí, q es el cociente y r es el residuo.

Demostración: estrategia

Supongamos que:

$$a = qb + r, \quad 0 \leq r < b.$$

Definimos el conjunto de divisores comunes de x y y :

$$\mathcal{D}(x, y) = \{d \in \mathbb{Z}^+ : d \mid x \wedge d \mid y\}.$$

Por la definición de MCD:

$$\text{mcd}(x, y) = \max \mathcal{D}(x, y).$$

Por tanto, basta probar que:

$$\mathcal{D}(a, b) = \mathcal{D}(b, r).$$

Lo haremos mediante doble inclusión.

Demostración: doble inclusión

Supongamos $a = qb + r$. Probaremos $\mathcal{D}(a, b) = \mathcal{D}(b, r)$ mediante doble inclusión.

$$\mathcal{D}(a, b) \subseteq \mathcal{D}(b, r)$$

Sea $c \in \mathcal{D}(a, b)$. Entonces:

$$c \mid a \wedge c \mid b.$$

Por divisibilidad:

$$\exists m, n \in \mathbb{Z} : a = mc \wedge b = nc.$$

Como $r = a - qb$:

$$r = mc - q(nc) = (m - qn)c.$$

Como $m - qn \in \mathbb{Z}$, $c \mid r$. Además, $c \mid b$; por tanto $c \in \mathcal{D}(b, r)$.

$$\mathcal{D}(b, r) \subseteq \mathcal{D}(a, b)$$

Sea $c \in \mathcal{D}(b, r)$. Entonces:

$$c \mid b \wedge c \mid r.$$

Por divisibilidad:

$$\exists n, s \in \mathbb{Z} : b = nc \wedge r = sc.$$

Como $a = qb + r$:

$$a = q(nc) + sc = (qn + s)c.$$

Como $qn + s \in \mathbb{Z}$, $c \mid a$. Además, $c \mid b$; por tanto $c \in \mathcal{D}(a, b)$.

$$\mathcal{D}(a, b) = \mathcal{D}(b, r) \quad \Rightarrow \quad \text{mcd}(a, b) = \text{mcd}(b, r).$$

De la invariancia al algoritmo de Euclides

El teorema permite sustituir un problema por otro equivalente:

$$\text{mcd}(a, b) = \text{mcd}(b, r), \quad a = qb + r, \quad 0 \leq r < b.$$

Aplicamos la misma idea de forma iterativa:

$$(a_0, b_0) \rightarrow (a_1, b_1) \rightarrow (a_2, b_2) \rightarrow \dots$$

donde, en cada paso,

$$a_i = q_i b_i + r_i, \quad (a_{i+1}, b_{i+1}) = (b_i, r_i).$$

Como $0 \leq r_i < b_i$, el segundo componente disminuye hasta llegar a cero.

Conclusión algorítmica

Repetir divisiones euclidianas hasta obtener residuo cero. El último divisor no nulo es el MCD.

Algoritmo de Euclides: pseudocódigo

El algoritmo de Euclides calcula el **máximo común divisor (MCD)** de dos enteros mediante divisiones sucesivas.

Su fundamento es la invariancia:

$$\text{mcd}(a, b) = \text{mcd}(b, r), \quad a = qb + r, \quad 0 \leq r < b.$$

Algoritmo de Euclides

Entrada: $a, b \in \mathbb{Z}$, no ambos cero

Salida: $d = \text{mcd}(a, b)$

Inicializar $a \leftarrow |a|$, $b \leftarrow |b|$

Repetir mientras $b \neq 0$:

 Dividir a entre b : $a = qb + r$, $0 \leq r < b$

 Actualizar $a \leftarrow b$

 Actualizar $b \leftarrow r$

Hasta que $b = 0$

Regresar $d \leftarrow a$